

Guía de Actividades Práctico-Experimentales Nro. 006

1. Datos Generales

Asignatura	Estructura de datos
Ciclo	3 A
Unidad	2
Resultado de aprendizaje de la unidad	Aplica los métodos de ordenación y búsqueda en la resolución de problemas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Título de la Práctica	Ordenación básica en Java: Burbuja, Selección e Inserción
Nombre del Docente	Andrés Roberto Navas Castellanos
Fecha	Jueves 20 de noviembre Viernes 21 de noviembre
Horario	07h30 – 10h30 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	5 horas
Estudiantes	- Fernando Sebastian Patiño Diaz - Ana Cristina Panamito Flores
Link del repositorio	https://github.com/Fede-dev11/Sorting_Taller5/tree/Taller6

2. Objetivo(s) de la Práctica:

- Ejecutar y analizar comparativamente los algoritmos de Burbuja, Selección e Inserción sobre casos de prueba, para determinar cuándo conviene cada uno en función de tamaño, grado de orden y duplicados.

3. Materiales y reactivos:

- Guía de pruebas con datasets y salidas esperadas.

4. Equipos y herramientas

- JDK OpenJDK (obligatorio).
- IDE: Visual Studio Code (extensión “Extension Pack for Java”) o IntelliJ IDEA Community.
- Sistema de control de versiones: Git; repositorio en GitHub.
- EVA/Moodle institucional: para entrega de evidencias.
- Herramientas de documentación: README Markdown, editor ofimático (Google Docs/LibreOffice/Word).

5. Procedimiento / Metodología

Enfoque metodológico: ABPr (Aprendizaje Basado en Proyectos). Inicio

- Presentación del objetivo comparativo y criterios de éxito.
- Formación de equipos (3–4) y revisión de la rúbrica.
- Creación de repo Git.
- Lineamientos de uso responsable de IA.

Desarrollo

- Paso 1. Instrumentación (obligatorio)
 - Añade contadores a tus algoritmos:
 - `comparisons++` al comparar dos claves,
 - `swaps++` al intercambiar posiciones.
 - Mide tiempo con ``System.nanoTime()`` sin imprimir durante la medición (las trazas distorsionan).
 - Ejecuta R repeticiones por caso (sug.: R=10), descarta las 3 primeras (calentamiento/JIT) y reporta la mediana de tiempo.
 - Aísla IO: carga CSV fuera de la medición; mide sólo el ordenamiento del array en memoria.
- Paso 2. Casos de prueba
 - Define clave de orden (p. ej., ``fechaHora`` en ``citas``, ``apellido`` en ``pacientes``, ``stock`` en ``inventario``).
 - Convierte a array de la clave (o a registros con ``Comparable`` por clave).
 - Ejecuta: Insertion, Selection, Bubble (con “corte temprano” en Burbuja).
 - Registra: n, %casi-ordenado, %duplicados, comparisons, swaps, tiempo(ns) (mediana de R-3 corridas).
- Paso 3. Análisis
 - Tablas comparativas por caso (n, orden, duplicados) y gráficos (tiempo vs. n; tiempo vs. %casi-ordenado).
 - Matriz de recomendación (reglas prácticas):
 - Casi ordenado + n pequeño/medio → Inserción gana (menos movimientos).
 - Muchos duplicados → Inserción tiende a mantener estabilidad útil; Selección hace $n(n-1)/2$ comparaciones siempre, con pocos swaps.
 - Inverso o aleatorio (n pequeño/educativo) → cualquiera, pero Burbuja penaliza; Selección constante en comparaciones; Inserción peor en inverso pero mejor si detecta localmente orden.

Cierre

- **Discusión guiada: ¿Cuándo conviene cada uno? ¿Qué sesgos introdujo la medición?**

Bubble Sort conviene solo cuando la lista está casi ordenada, porque su optimización de corte temprano permite terminar antes si no hay intercambios. También es útil para enseñar paso a paso cómo funcionan los intercambios. No es recomendable para listas grandes o muy desordenadas por su alto número de swaps.

Selection Sort es adecuado cuando se desea minimizar la cantidad de intercambios, ya que siempre realiza como máximo un swap por iteración. Es útil cuando mover elementos es



costoso, como en estructuras grandes. Sin embargo, no aprovecha si la lista está ordenada, ya que hace siempre el mismo número de comparaciones.

Insertion Sort conviene para listas pequeñas o parcialmente ordenadas, donde resulta muy eficiente en tiempo y requiere pocos movimientos. También es ideal cuando los datos se van insertando gradualmente. En listas grandes y muy desordenadas pierde rendimiento, pero sigue siendo mejor que Bubble Sort en la mayoría de casos.

Las listas tienen tamaños diferentes, lo cual favorece a unos algoritmos y perjudica a otros. Los comparadores utilizados (fecha, apellido o stock) no cuestan lo mismo, afectando el tiempo registrado. Los swaps implica que al mover objetos completos, haciendo que algoritmos los cuales van tener muchos intercambios (como Bubble Sort) los cual va hacer que parezcan peores. El estado inicial del dataset (ordenado, casi ordenado o aleatorio) altera drásticamente el rendimiento, especialmente en Insertion Sort. La carga de archivos y la optimización interna de la JVM también influyen en el tiempo de ejecución. Finalmente, cada algoritmo se ejecuta sobre distintos tipos de datos, lo que introduce variabilidad adicional en los resultados.

- Completar README e informe con evidencias y la matriz de recomendación.

6. Resultados esperados:

- Tabla por dataset: `n, tipo (aleat/casi-ord/dup/inverso), algoritmo, comparisons, swaps, tiempo\_mediana(ns)`.
- Gráficos (opcional): barras o líneas para tiempo y comparaciones.
- Matriz de recomendación (texto/tabla): “si casi ordenado y $n \leq 500 \rightarrow$ Inserción”, “si minimizar swaps $\rightarrow$ Selección”, etc.
- Capturas/Logs de ejecución (sin trazas durante medición).
- Código con instrumentación y scripts de generación de datasets (si aplica).

```
==== RESULTADOS BURBUJA ====
Burbuja - Citas          | Comparaciones: 4895   | Swaps: 2337   | Tiempo(ns): 4338256
Burbuja - Pacientes      | Comparaciones: 124344 | Swaps: 59452  | Tiempo(ns): 30731897
Burbuja - Inventario     | Comparaciones: 124750 | Swaps: 124750 | Tiempo(ns): 26749064
Burbuja - Citas Casi     | Comparaciones: 4047   | Swaps: 307    | Tiempo(ns): 529752

→ Menor número de swaps:
Burbuja - Citas Casi     | Comparaciones: 4047   | Swaps: 307    | Tiempo(ns): 529752

----- MENÚ -----
1. Burbuja en todas las listas
2. Insertion Sort en todas las listas
3. Selection Sort en todas las listas
4. Salir
Ingrese opción: |
```

```

==== RESULTADOS INSERTION ====
Insertion - Citas      | Comparaciones: 2433 | Swaps: 2337 | Tiempo(ns): 457886
Insertion - Pacientes  | Comparaciones: 59948 | Swaps: 59452 | Tiempo(ns): 8536447
Insertion - Inventario | Comparaciones: 124750 | Swaps: 124750 | Tiempo(ns): 2449884
Insertion - Citas Casi | Comparaciones: 406 | Swaps: 307 | Tiempo(ns): 96082

→ Menor número de swaps:
Insertion - Citas Casi | Comparaciones: 406 | Swaps: 307 | Tiempo(ns): 96082

----- MENÚ -----
1. Burbuja en todas las listas
2. Insertion Sort en todas las listas
3. Selection Sort en todas las listas
4. Salir
Ingrese opción: |

```

```

==== RESULTADOS SELECTION ====
Selection - Citas      | Comparaciones: 4950 | Swaps: 97 | Tiempo(ns): 631485
Selection - Pacientes  | Comparaciones: 124750 | Swaps: 486 | Tiempo(ns): 8558167
Selection - Inventario | Comparaciones: 124750 | Swaps: 250 | Tiempo(ns): 1777111
Selection - Citas Casi | Comparaciones: 4950 | Swaps: 5 | Tiempo(ns): 144433

→ Menor número de swaps:
Selection - Citas Casi | Comparaciones: 4950 | Swaps: 5 | Tiempo(ns): 144433

----- MENÚ -----
1. Burbuja en todas las listas
2. Insertion Sort en todas las listas
3. Selection Sort en todas las listas
4. Salir
Ingrese opción: |

```

7. Preguntas de Control:

- **¿Por qué imprimir trazas durante la medición distorsiona los tiempos?**

Imprimir trazas distorsiona los tiempos porque las operaciones de Entrada/Salida (I/O) son significativamente más lentas y costosas para el sistema que las operaciones internas de la CPU (como comparaciones y *swaps*)

- El tiempo de I/O se suma al tiempo de ejecución del algoritmo de ordenamiento.
- Esto hace que la medición resultante sea un reflejo del tiempo del algoritmo más el tiempo de impresión, lo que no permite comparar la eficiencia algorítmica pura.
- Por esta razón, la guía exige que la medición de tiempo se realice sin imprimir las trazas

- **Explica por qué Selección tiene comparaciones $n(n-1)/2$ sin importar el orden inicial.**

El algoritmo Selection (Ordenamiento por Selección) tiene una cantidad de comparaciones que es constante e independiente del estado inicial del arreglo (ordenado, inverso, o aleatorio).

El algoritmo opera en dos partes: una ordenada y una no ordenada. En cada pasada, debe escanear completamente la porción restante no ordenada para encontrar el elemento más pequeño (el mínimo).

- La primera pasada requiere $n-1$ comparaciones.
- La segunda pasada requiere $n-2$ comparaciones.
- Esto continúa hasta la última pasada que requiere 1 comparación.

La suma total de estas comparaciones es una serie aritmética que siempre resulta

en:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Dado que la búsqueda del mínimo siempre obliga a realizar esta secuencia fija de comparaciones, el número de comparaciones de Selección es siempre $O(n^2)$.

- **¿Por qué Inserción es competitivo en datos casi ordenados?**

Debido a que el algoritmo de Inserción es extremadamente eficiente con datos casi ordenados porque su costo operativo (el número de comparaciones y movimientos) depende directamente de cuán desordenado esté el arreglo. Cuando el arreglo está completamente ordenado (el mejor caso), Insertion Sort solo necesita hacer $n-1$ comparaciones (una por cada elemento para verificar que ya está en su lugar) y realiza cero swaps.

Dado que los datos casi ordenados se comportan de manera muy similar al mejor caso, Inserción mantiene su eficiencia cercana a, superando fácilmente la complejidad fija de los otros dos.

- **¿Qué papel juegan los duplicados en la estabilidad del resultado?**

Los duplicados son la prueba de la estabilidad de un algoritmo de ordenamiento.

Un algoritmo es estable si preserva el orden relativo original de dos o más elementos que tienen el mismo valor (duplicados).

- **Impacto de Duplicados:** Si tienes dos elementos idénticos, por ejemplo, dos registros con el valor 5 (5A y 5B, donde 5A aparece primero en la entrada), un algoritmo estable garantizará que 5A continúe antes de 5B en la salida.
- La guía menciona que Insertion Sort tiende a mantener la estabilidad. Esto es crucial en aplicaciones reales donde los registros contienen más información (por ejemplo, ordenar pacientes por apellido, pero mantener su orden original por fecha de ingreso si los apellidos son idénticos)

- **¿Por qué Burbuja con corte temprano mejora en “casi ordenado” pero no en “inverso”?**

El algoritmo de Burbuja con corte temprano (o *early exit*) incluye una optimización crítica: un *flag* booleano (como *swapped* en tu código) que se activa si ocurre algún intercambio de elementos durante una pasada completa.

En un arreglo "casi ordenado", la optimización es altamente efectiva:

- Dado que la mayoría de los elementos ya están en su lugar o muy cerca, solo se necesita un puñado de intercambios (swaps) para corregir las desalineaciones.
- Una vez que estos pocos swaps iniciales han ocurrido, el algoritmo realiza una o dos pasadas de verificación completas. En estas pasadas, el *flag* *swapped* permanecerá falso.
- El corte temprano se activa, deteniendo la ejecución. Esto reduce drásticamente el número total de pasadas y comparaciones.



8. Evaluación

Criterio	4 – Excelente	3 – Bueno	2 – Básico	1 – Insuficiente	Pts
Instrumentación (contadores + tiempo)	Corrección y limpieza; medición sin IO/impresiones	Menor detalle	Parcial	No funcional	2.5
Diseño experimental	$R \geq 10$, descarta 3 corridas, mediana; casos variados	Algún ajuste menor	Parcial	Inadecuado	2.0
Ejecución y datos	Tablas completas por dataset	Tablas con huecos	Datos escasos	Sin datos	2.0
Análisis y matriz	Conclusiones claras y justificadas	Aceptables	Superficiales	Ausentes	2.5
Entrega y código	README/Informe claros; código limpio	Aceptable	Pobre	Deficiente	1.0

9. Bibliografía

- [1] OpenDSA Project, “Sorting and Searching Modules,” Virginia Tech, 2021–2024 (REA con visualizaciones y ejercicios).
- [2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.
- [3] Oracle, “Java SE 17–21 Documentation: `Arrays`, Collections, and I/O (`java.nio.file`), and benchmarking notes,” 2021–2025.
- [4] OpenJDK, “JMH – Java Microbenchmark Harness: Samples and Guidance,” 2020–2025 (guía práctica de mediciones reproducibles).



10. Elaboración y Aprobación

Elaborado por	Andrés R Navas Castellanos Docente	 Firmado electrónicamente por: ANDRES ROBERTO NAVAS CASTELLANOS Validar únicamente con FirmaEC
Revisado por Solo si es realizado en laboratorios	Luis Sinche Técnico Docente	No Aplica
Aprobado por	Edison L Coronel Romero Director de Carrera	